第 13 天 • 11/14 • 今日知识点回顾

merge 连接：how 四种取值、连接键对不上怎么办

目标：说清 left / inner / outer / right 四种连接的差别，并能解释「为什么 left join 之后行数变多了」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 说清 left / inner / outer / right 四种连接的差别，并能解释「为什么 left join 之后行数变多了」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 52 分钟（边看边在本机敲一遍）

P31	Pandas2数据合并与对比	约 21 分钟
P32	【数据的合并和分组聚合】02数据合并	约 19 分钟
P35	【数据的合并和分组聚合】05数据的索引学习	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch6.rst`

第六章 连接：merge / join / concat 的区别与 on/left_on/right_on

知识点 1/7 • merge 的四个 how，差别只在「留谁的行」

- 一句话：inner 交集、left 留左表全部、right 留右表全部、outer 并集。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
for how in ['inner', 'left', 'right', 'outer']:
m = df.merge(u[['user_id', 'gender', 'age']], on='user_id', how=how)
print(how, len(m), m['gender'].isna().sum())
```

★ 本机实测：left 连接后 1163 行，其中 39 行的 gender 是 nan ——说明订单里有 39 个 user_id 在 users 表里查不到。这 39 行就是「脏数据」，清洗时要单独交代（是用户被删了，还是订单表的 user_id 写错了）。

这一页只讲「merge 的四个 how，差别只在「留谁的行」」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • left join 行数变多 = 右表在连接键上不唯一

- 一句话：右表一个键对应多行，左表一行就会被复制成多行 —— 这是最常见的「数据翻倍」事故。
- 代码：

```
print('右表 user_id 唯一? ', u['user_id'].is_unique)      # True
dup = pd.DataFrame({'user_id': [1001, 1001], 'tag': ['新客', '老客']})
m = df.head(3).merge(dup, on='user_id', how='left')
print(len(m))                                             # 4 — 1001 那一行变两行
```

★ 连接前先跑 df['键'].is_unique 和 df['键'].duplicated().sum()。

- 多对多连接会让汇总金额翻倍，而且不报错 —— 这是最贵的一类 bug。

知识点 3/7 • 连接键名字不同：left_on / right_on

- 一句话：左表叫 user_id、右表叫 uid 时，用 left_on='user_id', right_on='uid'。
- 代码：

```
u2 = u.rename(columns={'user_id': 'uid'})
m = df.merge(u2[['uid', 'gender']], left_on='user_id', right_on='uid', how='left')
print(m.columns.tolist()[:6])
print(len(m))
```

★ 这样连完 user_id 和 uid 两列都在，值一样 —— 记得 drop 掉一列，否则后面groupby 会出现两列重复的键，导出 CSV 时业务方会问「怎么有两个 ID」。

知识点 4/7 • indicator 参数：一眼看出哪行没连上

- 一句话：merge(..., indicator=True) 会多一列 _merge，值是 both / left_only / right_only。
- 代码：

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left', indicator=True)
print(m['_merge'].value_counts().to_dict())
miss = m[m['_merge'] == 'left_only']
print('连不上的订单数：', len(miss), '涉及金额：', round(miss['amount'].sum(), 2))
```

★ 连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。

知识点 5/7 • validate：让 merge 主动检查「一对几」

- 一句话：validate='one_to_one' / 'one_to_many' / 'many_to_one'，不符合直接报错。
- 代码：

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left',
validate='many_to_one')       # 左多右一，符合预期
print(len(m))
# validate='one_to_one' 会报
# MergeError: Merge keys are not unique in either left or right dataset
```

★ 加上 validate 之后，「不小心连成多对多」会在当场炸掉，而不是三天后发现报表数字翻倍。生产脚本里建议每个 merge 都带。

知识点 6/7 • concat：纵向摞，不是横向连

- 一句话：pd.concat([df1, df2], ignore_index=True) 把两个同结构的表摞起来。
- 代码：

```
a = df.head(3)
b = df.tail(3)
print(len(pd.concat([a, b], ignore_index=True)))      # 6
print(pd.concat([a, b]).index.tolist())                # 0,1,2,1160,1161,1162 重复
```

★ 不加 ignore_index=True，索引会重复（0,1,2,1160,1161,1162），后面df.loc[0] 会返回两行 —— 「莫名其妙多了几行」的另一个来源。

知识点 7/7 • 真数据：把用户信息接到订单上，做分性别汇总

- 一句话：merge 之后 groupby 才有意义 —— 单表只能按已有列分组。
- 代码：

```
m = df.merge(u[['user_id', 'gender', 'age']], on='user_id', how='left')
paid = m[m['status'] == 'paid']
g = (paid.groupby('gender', dropna=False)
.agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
.round(2))
print(g.to_string())
```

- （users 表的 gender 有六种写法：男 / 女 / M / F / 未知 / 空。
- 直接 groupby 会得到六行 —— 第 16 天会专门讲怎么归并成两行 + 一行未知。）

这一页只讲「真数据：把用户信息接到订单上，做分性别汇总」。讲完就翻页，下一页是另一个知识点。

代码速查：今天这几段，要在本机跑通（1/5）

写 merge_practice.py，每题打印行数并解释：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
```

1) 四种 how 的行数与缺失数

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
for how in ['inner', 'left', 'right', 'outer']:
```

```
    m = df.merge(u[['user_id', 'gender']], on='user_id', how=how)
```

```
    print(f'{how:6s} rows={len(m):5d} gender_na={m["gender"].isna().sum():4d}')
```

2) indicator 找出连不上的订单

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left', indicator=True)
```

代码速查：今天这几段，要在本机跑通（3/5）

```
print(m['_merge'].value_counts().to_dict())
```

miss = m[m['_merge'] == 'left_only']

```
print('连不上：', len(miss), '金额合计', round(miss['amount'].sum(), 2))
```

3) validate 检查一对多

```
ok = df.merge(u[['user_id', 'gender']], on='user_id', how='left', validate='many_to_one')
```

```
print('many_to_one 通过，行数', len(ok))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

4) 接上用户表后按性别汇总

paid = ok[ok['status'] == 'paid']

```
print(paid.groupby('gender', dropna=False)
```

```
          .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
```

.round(2).to_string())

验收（做到才算过关）：

```
① 第 1 题四个 how 的行数都能解释（inner < left ≤ outer）；
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

② 连不上的订单数是 39，金额合计能报出来；

③ 能说清「为什么 left join 可能比左表行数多」，并举一个具体例子。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 本机实测：left 连接后 1163 行，其中 39 行的 gender 是 nan ——说明订单里有 39 个 user_id 在 users 表里查不到。这 39 行就是「脏数据」，清洗时要单独交代（是用户被删了，还是订单表的 user_id 写错了）。

多对多连接会让汇总金额翻倍，而且不报错 —— 这是最贵的一类 bug。

★ 连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：连接前先跑 df['键'].is_unique 和 df['键'].duplicated().sum()。
- [ ] 我能不看资料写出：连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。
- [ ] 我能不看资料写出：加上 validate 之后，「不小心连成多对多」会在当场炸掉，而不是三天后发现报表数字翻倍。生产脚本里建议每个 merge 都带。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 14 天 • ★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-14-第13天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data